

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

ΕΞΑΜΗΝΙΑΙΑ ΑΣΚΗΣΗ 2023

ΕΠΕΞΕΡΓΑΣΤΗΣ MIPS ΑΠΛΟΥ ΚΥΚΛΟΥ

ΣΤΟΙΧΕΙΑ ΦΟΙΤΗΤΗ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ : ΑΘΑΝΑΣΙΟΥ ΒΑΣΙΛΕΙΟΣ ΕΥΑΓΓΕΛΟΣ

ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ : 19390005

ΕΞΑΜΗΝΟ ΦΟΙΤΗΤΗ : 8^ο

ΚΑΤΑΣΤΑΣΗ ΦΟΙΤΗΤΗ : ΠΡΟΠΤΥΧΙΑΚΟ

ΠΡΟΓΡΑΜΜΑ ΣΠΟΥΔΩΝ : ΠΑΔΑ

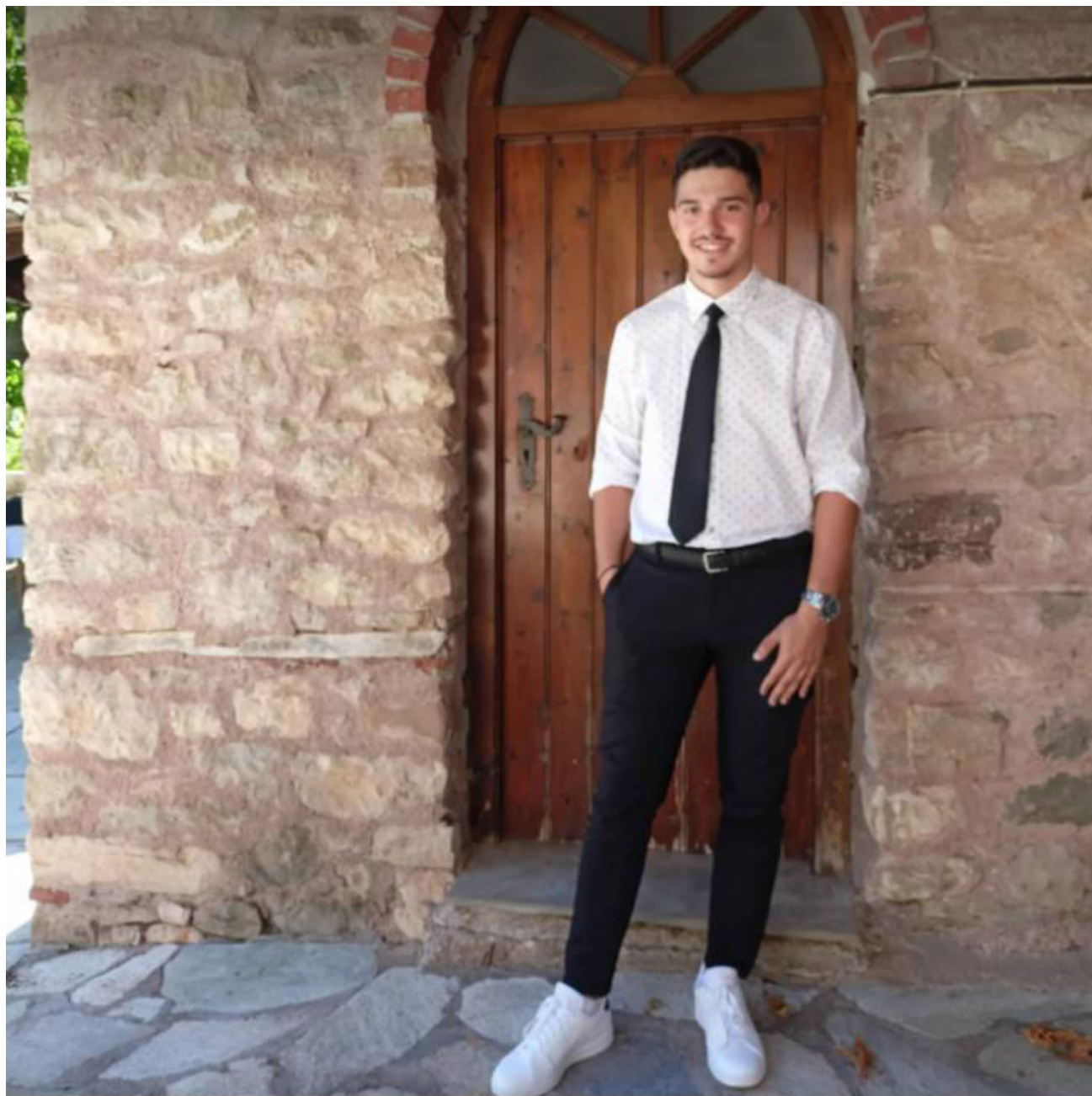
ΤΜΗΜΑ ΘΕΩΡΙΑΣ : ΤΜΗΜΑ 2 ΚΑΡΚΑΖΗΣ ΠΑΝΑΓΙΩΤΗΣ

ΥΠΕΥΘΥΝΟΙ ΘΕΩΡΙΑΣ : ΒΟΓΙΑΤΖΗΣ ΙΩΑΝΝΗΣ – ΚΑΡΚΑΖΗΣ ΠΑΝΑΓΙΩΤΗΣ

ΗΜΕΡΟΜΗΝΙΑ ΠΑΡΑΔΟΣΗΣ : 15/9/2023

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΦΩΤΟΓΡΑΦΙΑ ΦΟΙΤΗΤΗ :

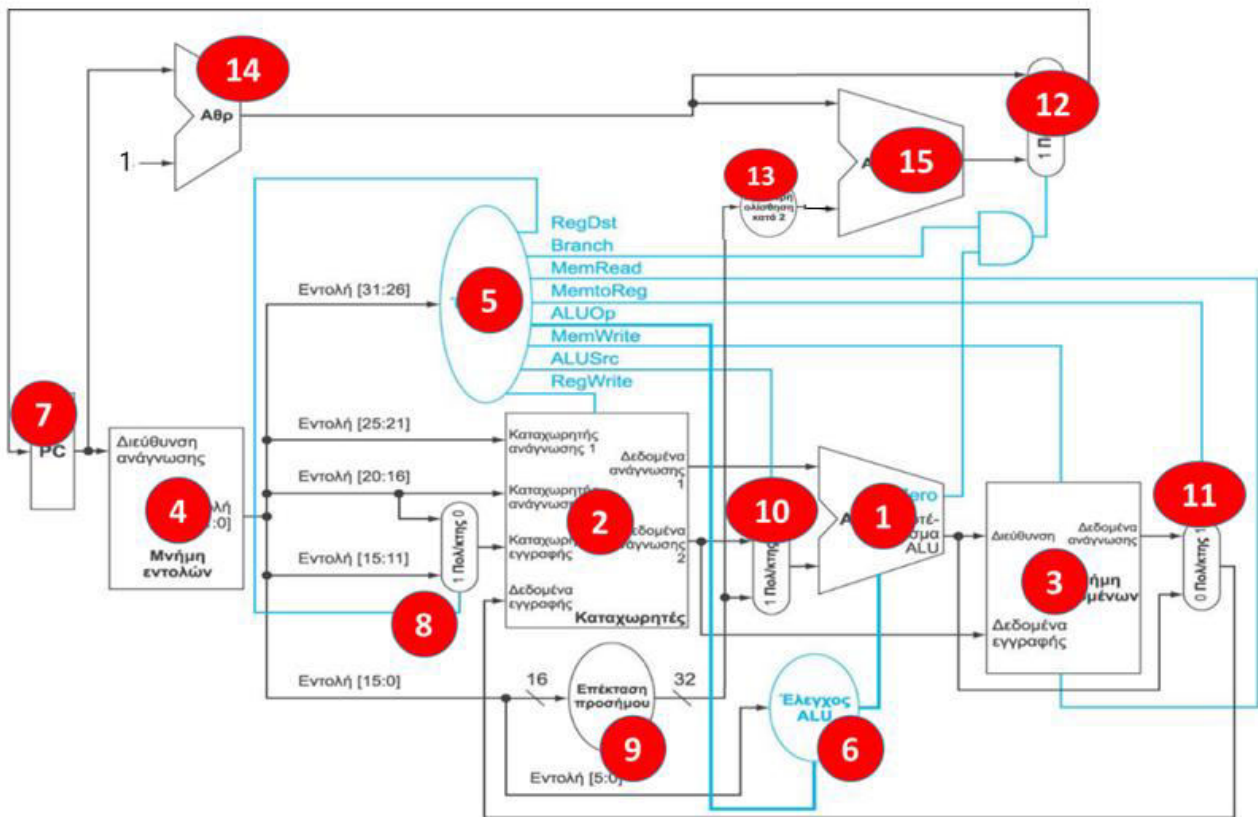


ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΠΕΡΙΕΧΟΜΕΝΑ

<u>1. Περιγραφή του κυκλώματος</u>	<u>(ΣΕΛΙΔΕΣ 4-6)</u>
<u>2. Κώδικας του κυκλώματος και testbench</u>	<u>(ΣΕΛΙΔΕΣ 7-53)</u>
<u>3. Screenshot από την εκτέλεση</u>	<u>(ΣΕΛΙΔΕΣ 54-56)</u>

1. Περιγραφή του κυκλώματος



Εικόνα 1. Επεξεργαστής MIPS απλού κύκλου

1. ALU

Η Αριθμητική Λογική Μονάδα (ALU) δέχεται δύο εισόδους και εκτελεί πράξεις αριθμητικές (πρόσθεση, αφαίρεση) και λογικές (AND, OR, XOR κλπ.) bit προς bit. Το ποια πράξη θα εκτελέσει καθορίζεται από την έξοδο της Μονάδας Ελέγχου ALU, που ουσιαστικά στέλνει ένα σήμα ελέγχου των 2 bit που σηματοδοτεί την πράξη που θα εκτελεστεί στην ALU.

Αν πρόκειται για αριθμητική πράξη, τότε η έξοδος είναι το “Αποτέλεσμα ALU”, διαφορετικά, αν πρόκειται για λογική, η έξοδος είναι το “Zero”.

2. Register File

Το Register File περιέχει τους καταχωρητές, όπου αποθηκεύονται τα δεδομένα και είναι προσβάσιμα για εγγραφή και ανάγνωση. Πιο συγκεκριμένα, περιέχει 2 καταχωρητές ανάγνωσης και έναν καταχωρητή εγγραφής. Επίσης, περιέχει 2 δεδομένα ανάγνωσης των 32-bit που αποτελούν την έξοδο των 2 καταχωρητών ανάγνωσης. Ουσιαστικά οι καταχωρητές περιέχουν 5-bit που καθορίζουν τον αριθμό του καταχωρητή, ενώ τα δεδομένα περιέχουν το περιεχόμενο των καταχωρητών, εξού και η χωρητικότητα των 32-bit πληροφορίας.

Αντίστοιχα, και για εγγραφή ισχύει το ίδιο με την διαφορά ότι εγγραφή πραγματοποιείται μόνο όταν το σήμα ελέγχου “RegWrite” της Μονάδας Ελέγχου είναι 1.

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

3. Μνήμη Δεδομένων

Η μνήμη δεδομένων περιέχει μία διεύθυνσης μνήμης και τα δεδομένα που αποθηκεύτηκαν στον έναν καταχωρητή ανάγνωσης του Register File. Επίσης, υπάρχουν 2 σήματα ελέγχου για ανάγνωση και εγγραφή που συνδέονται με την Μονάδα Ελέγχου, MemRead και MemWrite, αντίστοιχα. Η έξοδος της μνήμης δεδομένων χρησιμοποιείται για εγγραφή στο Register File, εφόσον, έχουν ενεργοποιηθεί τα κατάλληλα σήματα ελέγχου από την Μονάδα Ελέγχου.

4. Μνήμη Εντολών

Η μνήμη εντολών περιέχει μία διεύθυνση μνήμης για είσοδο και την εντολή που αντιστοιχεί σε αυτή την διεύθυνση, για έξοδο.

5. Μονάδα Ελέγχου

Η μονάδα ελέγχου περιέχει κατάλληλα σήματα ελέγχου, ώστε να καθοριστούν οι αντίστοιχες λειτουργίες που επιχειρούν να εκτελέσουν άλλα εξαρτήματα του επεξεργαστή.

6. Μονάδα Ελέγχου ALU

Κάθε εντολή που ανήκει σε κάποια κατηγορία π.χ. τύπου R (add, addi ...) έχει και ένα bit opcode, το οποίο αποτελεί το κύριο σήμα ελέγχου για την ALU, ώστε να γνωρίζει ποια εντολή θα εκτελέσει.

7. PC

Ο program counter περιέχει την διεύθυνση της επόμενης εντολής που θα εκτελεστεί και είναι ουσιαστικά ένας απλός καταχωρητής 32-bit.

8. 5-πλός πολυπλέκτης 2-σε-1

Ο πολυπλέκτης αποτελεί ένα ακολουθιακό κύκλωμα που, ανάλογα το σήμα ελέγχου μεταβιβάζει και την αντίστοιχη είσοδο, στην έξοδο. Ανάλογα με το σήμα ελέγχου RegDst της Μονάδας Ελέγχου, ο πολυπλέκτης μεταβιβάζει τα αντίστοιχα 5-bit πληροφορίας από την εντολή που εισήχθη από την μνήμη εντολών, στον καταχωρητή εγγραφής του Register File.

9. Μονάδα επέκτασης προσήμου 16-σε-32

Η μονάδα επέκτασης προσήμου, μετατρέπει τον 16-bit αριθμό σε 32-bit, ώστε να γίνει η πρόσθεση στον αθροιστή των 32-bits.

10. 32-πλος πολυπλέκτης 2-σε-1

Ο πολυπλέκτης αυτός μεταβιβάζει ανάλογα το bit ελέγχου «ALUSrc» της μονάδας ελέγχου, την αντίστοιχη πληροφορία στην είσοδο της ALU.

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

11. 32-πλος πολυπλέκτης 2-σε-1

Ο πολυπλέκτης αυτός μεταβιβάζει ανάλογα το bit ελέγχου «MemtoReg» της μονάδας ελέγχου, την αντίστοιχη πληροφορία στα δεδομένα εγγραφής του Register File.

12. 32-πλος πολυπλέκτης 2-σε-1

Ο πολυπλέκτης αυτός μεταβιβάζει ανάλογα το αποτέλεσμα Branch AND Zero της μονάδας ελέγχου και της ALU αντίστοιχα, την αντίστοιχη πληροφορία στην είσοδο του Program Counter.

13. Μονάδα Ολίσθησης αριστερά κατά 2 (32-bits)

Η μονάδα ολίσθησης, μετακινεί την πληροφορία κατά 2-bits αριστερά, ώστε να μην προκύψει το φαινόμενο υπερχείλισης στην πρόσθεση.

14. Αθροιστής 32 bits

Ο αθροιστής προσθέτει την διεύθυνση που περιέχει ο program counter κατά 1 για την επόμενη εντολή που θα εκτελεστεί.

15. Αθροιστής 32 bits

Ο αθροιστής που προσθέτει την διεύθυνση του program counter αυξημένη κατά 1 με την πληροφορία ολισθημένη κατά 2 bit αριστερά.

Πύλη AND 2 εισόδων

Η πύλη AND που έχει σαν είσοδο το σήμα ελέγχου Branch της μονάδας ελέγχου και του σήματος εξόδου Zero της ALU.

MIPS

Η κατάλληλη σύνδεση των προαναφερόμενων εξαρτημάτων αποτελεί τον επεξεργαστή MIPS απλού κύκλου. Ουσιαστικά, η λειτουργία του MIPS περιγράφεται ως εξής :

1. Ανάκτηση της διεύθυνσης της εντολής από τον program counter
2. Αποκωδικοποίηση της εντολής από την μνήμη εντολών και μεταβίβαση στους καταχωρητές του Register File και στην είσοδο της Μονάδας Ελέγχου
3. Πραγματοποίηση των κατάλληλων ελέγχων για μεταβίβαση της πληροφορίας στις εισόδους της ALU
4. Εκτέλεση της εντολής
5. Απόθηκευση στην μνήμη δεδομένων
6. Υπολογισμός της διεύθυνσης της επόμενης προς εκτέλεση εντολής.

2. Κώδικας του κυκλώματος και testbench

1. ALU

alu.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;
USE ieee.std_logic_signed.all;

ENTITY alu IS
    PORT (
        operationSig    : IN std_logic_vector(3 DOWNTO 0);
        ALU_input1       : IN std_logic_vector(31 DOWNTO 0);
        ALU_input2       : IN std_logic_vector(31 DOWNTO 0);
        ALU_output        : OUT std_logic_vector(31 DOWNTO 0);
        zeroSig          : OUT std_logic);
END alu;

ARCHITECTURE behavioral OF alu IS
BEGIN
    PROCESS(operationSig)
        VARIABLE tempInt : integer := 0;
    BEGIN
        CASE operationSig IS
            WHEN "0001" => -- addi
                zeroSig <= '0';
                tempInt := to_integer(signed(ALU_input1) + signed(ALU_input2));
                ALU_output <= std_logic_vector(to_signed(tempInt, 32));
            WHEN "0100" => -- add
                zeroSig <= '0';
                tempInt := to_integer(signed(ALU_input1) + signed(ALU_input2));
                ALU_output <= std_logic_vector(to_signed(tempInt, 32));
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
WHEN "0101" => -- sub
    tempInt := to_integer(signed(ALU_input1) - signed(ALU_input2));
    IF tempInt = 0 THEN
        zeroSig <= '1';
    ELSE
        zeroSig <= '0';
    END IF;
    ALU_output <= std_logic_vector(to_signed(tempInt, 32));
WHEN "1000" => -- bne
    tempInt := to_integer(signed(ALU_input1) - signed(ALU_input2));
    IF tempInt = 0 THEN
        zeroSig <= '1';
    ELSE
        zeroSig <= '0';
    END IF;
    ALU_output <= ALU_input1 XOR ALU_input2;
WHEN others =>
    zeroSig <= '0';
    ALU_output <= (others => '0');
END CASE;
END PROCESS;
END behavioral;
```

alu_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY alu_tb IS
END alu_tb;

ARCHITECTURE testbench OF alu_tb IS

    COMPONENT alu
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

PORT (

```
    operationSig    : IN std_logic_vector(3 DOWNTO 0);
    ALU_input1      : IN std_logic_vector(31 DOWNTO 0);
    ALU_input2      : IN std_logic_vector(31 DOWNTO 0);
    ALU_output      : OUT std_logic_vector(31 DOWNTO 0);
    zeroSig         : OUT std_logic);
```

END COMPONENT;

SIGNAL operationSig_tb : std_logic_vector(3 DOWNTO 0);

SIGNAL ALU_input1_tb, ALU_input2_tb, ALU_output_tb :
std_logic_vector(31 DOWNTO 0);

SIGNAL zeroSig_tb : std_logic;

BEGIN

```
Comp_Connection : alu PORT MAP (operationSig => operationSig_tb,
                                ALU_input1  => ALU_input1_tb,
                                ALU_input2  => ALU_input2_tb,
                                zeroSig     => zeroSig_tb,
                                ALU_output  => ALU_output_tb);
```

PROCESS

BEGIN

-- addi

operationSig_tb <= "0001";

ALU_input1_tb <= "000000000000000000000000000001111";

ALU_input2_tb <= "00000000000000000000000000000001";

wait for 10 ns;

-- add

operationSig_tb <= "0100";

ALU_input1_tb <= "000000000000000000000000000001111";

ALU_input2_tb <= "00000000000000000000000000000001";

wait for 10 ns;

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
-- sub (zero)
operationSig_tb <= "0101";
ALU_input1_tb    <= "000000000000000000000000000001111";
ALU_input2_tb    <= "000000000000000000000000000001111";
wait for 10 ns;

-- sub (non zero)
operationSig_tb <= "0101";
ALU_input1_tb    <= "000000000000000000000000000001111";
ALU_input2_tb    <= "000000000000000000000000000000001";
wait for 10 ns;

-- bne (equal)
operationSig_tb <= "1000";
ALU_input1_tb    <= "000000000000000000000000000001111";
ALU_input2_tb    <= "000000000000000000000000000001111";
wait for 10 ns;

-- bne (not equal)
operationSig_tb <= "1000";
ALU_input1_tb    <= "000000000000000000000000000001111";
ALU_input2_tb    <= "000000000000000000000000000000001";
wait for 10 ns;

END PROCESS;

END testbench;
```

2. Register File

register_file.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
USE ieee.numeric_std.ALL;
```

```
ENTITY register_file IS
```

```
    PORT (
```

```
        RF_clock      : IN std_logic;
        RF_reset      : IN std_logic;
        readReg1       : IN std_logic_vector(4 DOWNTO 0);
        readReg2       : IN std_logic_vector(4 DOWNTO 0);
        writeRegIn     : IN std_logic_vector(4 DOWNTO 0);
        writeData      : IN std_logic_vector(31 DOWNTO 0);
        RegWrite       : IN std_logic;
        readData1      : OUT std_logic_vector(31 DOWNTO 0);
        readData2      : OUT std_logic_vector(31 DOWNTO 0));
```

```
END register_file;
```

```
ARCHITECTURE behavioral OF register_file IS
```

```
    TYPE regArray IS ARRAY(0 TO 15) OF std_logic_vector(31 DOWNTO 0);
```

```
    SIGNAL register_file : regArray;
```

```
BEGIN
```

```
    PROCESS(RF_clock, RegWrite, RF_reset)
```

```
    BEGIN
```

```
        IF rising_edge(RF_clock) THEN
```

```
            IF RegWrite = '1' THEN
```

```
                register_file(to_integer(unsigned(writeRegIn))) <= writeData;
```

```
            END IF;
```

```
        END IF;
```

```
            readData1 <= register_file(to_integer(unsigned(readReg1)));
```

```
            readData2 <= register_file(to_integer(unsigned(readReg2)));
```

```
        IF RF_reset = '1' THEN
```

```
            readData1 <= (others => '0');
```

```
            readData2 <= (others => '0');
```

```
        END IF;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
END PROCESS;
END behavioral;
```

register_file_tb.vhd

[illegible]

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
writeRegIn_tb,

readReg1 => readReg1_tb,
readReg2 => readReg2_tb,
writeRegIn =>

writeData => writeData_tb,
RegWrite => RegWrite_tb,
readData1 => readData1_tb,
readData2 => readData2_tb);

clock_process : PROCESS
    BEGIN
        RF_clock_tb <= '0';
        wait for 5 ns;
        RF_clock_tb <= '1';
        wait for 5 ns;
    END PROCESS;

stim_process : PROCESS
    BEGIN
        -- Perform write operation
        RF_reset_tb <= '0';
        writeRegIn_tb <= "00001";
        writeData_tb <= "11110000111100001111000011110000";
        RegWrite_tb <= '1';
        WAIT FOR 10 ns;

        -- Perform read operation
        readReg1_tb <= "00001";
        readReg2_tb <= "00010";
        RegWrite_tb <= '0';
        WAIT FOR 10 ns;

        -- Reset the register file
        RF_reset_tb <= '1';
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
        WAIT FOR 10 ns;

        RF_reset_tb <= '0';

        WAIT;

        -- End the simulation

        WAIT;

    END PROCESS;

END testbench;
```

3. Μνήμη Δεδομένων

data_memory.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY data_memory IS
    PORT (
        writeData    : IN std_logic_vector(31 DOWNTO 0);
        addr         : IN std_logic_vector(31 DOWNTO 0);
        MemRead      : IN std_logic;
        MemWrite     : IN std_logic;
        readData     : OUT std_logic_vector(31 DOWNTO 0));
END data_memory;

ARCHITECTURE behavioral OF data_memory IS
    TYPE regArray IS ARRAY(0 TO 15) OF std_logic_vector(31 DOWNTO 0);
    SIGNAL data_memory : regArray := (others => (others => '0'));
    SIGNAL address : integer;

BEGIN

    address <= to_integer(unsigned(writeData)) WHEN
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```

        (to_integer(unsigned(writeData)) >= 0) ELSE 0;
    data_memory(address) <= writeData WHEN MemWrite = '1';
    readData <= data_memory(address) WHEN MemRead = '1';

END behavioral;
```

data_memory_tb.vhd

[illegible]

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

PROCESS

BEGIN

writeData_tb <= (others => '0');

addr_tb <= (others => '0');

MemRead_tb <= '0';

MemWrite_tb <= '0';

wait for 10 ns;

writeData_tb <= "11110000111100001111000011110000";

addr_tb <= "00000000000000000000000000000001";

MemWrite_tb <= '1';

wait for 10 ns;

addr_tb <= "00000000000000000000000000000001";

MemRead_tb <= '1';

wait for 10 ns;

writeData_tb <= "00001111000011110000111100001111";

addr_tb <= "00000000000000000000000000000010";

MemWrite_tb <= '1';

wait for 10 ns;

addr_tb <= "00000000000000000000000000000010";

MemRead_tb <= '1';

wait for 10 ns;

END PROCESS;

END testbench;

4. Μνήμη Εντολών

instructions_memory.vhd

LIBRARY ieee;

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
USE ieee.std_logic_1164.all;

USE ieee.numeric_std.all;

ENTITY instructions_memory IS
    PORT (
        readAddr      : IN std_logic_vector(31 DOWNTO 0);
        instrOut       : OUT std_logic_vector(31 DOWNTO 0));
END instructions_memory;

ARCHITECTURE behavioral OF instructions_memory IS
    TYPE regArray IS ARRAY(0 TO 15) OF std_logic_vector(31 DOWNTO 0);
    SIGNAL instructions_memory : regArray := (
        "00100000000000110000000000000001", -- addi $3, $0, 1
        "00100000000000101000000000000001", -- addi $3, $0, 1
        "0000000000110000000011001000000000", -- L1: add $6, $3, $0
        "10101100100001100000000000000000", -- sw $6, 0($4)
        "00100000011000110000000000000000", -- addi $3, $3, 1
        "00100000010000100000000000000000", -- addi $4, $4, 1
        "00100000010100101111111111111111", -- addi $5, $5, -1
        "00010100101000001111111111101010", -- bne $5,$0,L1
        others => (others => '0')
    );

    SIGNAL instrAddr : integer;
    SIGNAL instruction : std_logic_vector(31 DOWNTO 0);

BEGIN
    instrAddr <= to_integer(unsigned(readAddr)) WHEN
    (to_integer(unsigned(readAddr)) >= 0) ELSE 0;

    instruction <= instructions_memory(instrAddr) WHEN (instrAddr >= 0)
    ELSE std_logic_vector(to_signed(-1, 32));

    instrOut <= instruction;
END behavioral;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

instructions_memory_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY instructions_memory_tb IS
END instructions_memory_tb;

ARCHITECTURE testbench OF instructions_memory_tb IS

    COMPONENT instructions_memory
    PORT (
        readAddr      : IN std_logic_vector(31 DOWNTO 0);
        instrOut       : OUT std_logic_vector(31 DOWNTO 0));
    END COMPONENT;

    SIGNAL readAddr_tb, instrOut_tb : std_logic_vector(31 DOWNTO 0);

BEGIN

    Comp_Connection : instructions_memory PORT MAP (readAddr =>
readAddr_tb,
                                                    instrOut => instrOut_tb);

PROCESS
BEGIN
    readAddr_tb <= "00000000000000000000000000000001"; -- Read address 1
    wait for 10 ns;

    readAddr_tb <= "00000000000000000000000000000010"; -- Read address 2
    wait for 10 ns;

    readAddr_tb <= "00000000000000000000000000000011"; -- Read address 3
    wait for 10 ns;

END PROCESS;

END testbench;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

5. Μονάδα Ελέγχου

control_unit.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY control_unit IS
    PORT (
        CU_clock   : IN std_logic;
        Instr      : IN std_logic_vector(5 DOWNTO 0);
        RegDst     : OUT std_logic;
        Branch     : OUT std_logic;
        MemRead    : OUT std_logic;
        MemtoReg   : OUT std_logic;
        ALUOp      : OUT std_logic_vector(1 DOWNTO 0);
        MemWrite   : OUT std_logic;
        ALUSrc     : OUT std_logic;
        RegWrite   : OUT std_logic);
END control_unit;

ARCHITECTURE behavioral OF control_unit IS
BEGIN
    PROCESS (CU_clock, Instr)
    BEGIN
        IF rising_edge(CU_clock) THEN
            CASE Instr IS
                -- add, sub
                WHEN "000000" =>
                    RegDst <= '1';
                    Branch <= '0';
                    MemRead <= '0';
                    MemtoReg <= '0';
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
ALUOp <= "10";
MemWrite <= '0';
ALUSrc <= '0';
RegWrite <= '1';
--addi
WHEN "001000" =>
    RegDst <= '0';
    Branch <= '0';
    MemRead <= '0';
    MemtoReg <= '0';
    ALUOp <= "00";
    MemWrite <= '0';
    ALUSrc <= '1';
    RegWrite <= '1';
--lw
WHEN "100011" =>
    RegDst <= '0';
    Branch <= '0';
    MemRead <= '1';
    MemtoReg <= '1';
    ALUOp <= "00";
    MemWrite <= '0';
    ALUSrc <= '1';
    RegWrite <= '1';
--sw
WHEN "101011" =>
    RegDst <= 'X';
    Branch <= '0';
    MemRead <= '0';
    MemtoReg <= 'X';
    ALUOp <= "00";
    MemWrite <= '1';
    ALUSrc <= '1';
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
    RegWrite <= '0';
--bne
WHEN "000101" =>
    RegDst <= 'X';
    Branch <= '1';
    MemRead <= '0';
    MemtoReg <= 'X';
    ALUOp <= "01";
    MemWrite <= '0';
    ALUSrc <= '0';
    RegWrite <= '0';
WHEN others =>
    RegDst <= '0';
    Branch <= '0';
    MemRead <= '0';
    MemtoReg <= '0';
    ALUOp <= "00";
    MemWrite <= '0';
    ALUSrc <= '0';
    RegWrite <= '0';
END CASE;
END IF;
END PROCESS;
END behavioral;
```

control_unit_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY control_unit_tb IS
END control_unit_tb;

ARCHITECTURE testbench OF control_unit_tb IS
```


ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
clock_process : PROCESS
```

```
BEGIN
```

```
    CU_clock_tb <= '0';
```

```
    wait for 5 ns;
```

```
    CU_clock_tb <= '1';
```

```
    wait for 5 ns;
```

```
END PROCESS;
```

```
stim_process: PROCESS
```

```
BEGIN
```

```
    -- add, sub
```

```
    Instr_tb <= "000000";
```

```
    wait for 10 ns;
```

```
    -- addi
```

```
    Instr_tb <= "001000";
```

```
    wait for 10 ns;
```

```
    -- lw
```

```
    Instr_tb <= "100011";
```

```
    wait for 10 ns;
```

```
    -- sw
```

```
    Instr_tb <= "101011";
```

```
    wait for 10 ns;
```

```
    -- bne
```

```
    Instr_tb <= "000101";
```

```
    wait for 10 ns;
```

```
    -- default
```

```
    Instr_tb <= "111111";
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
wait for 10 ns;  
END PROCESS;  
END testbench;
```

6. Μονάδα Ελέγχου ALU

alu_control.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
  
ENTITY alu_control IS  
    PORT (  
        ALUOp      : IN std_logic_vector(1 DOWNTO 0);  
        Instr      : IN std_logic_vector(5 DOWNTO 0);  
        ALUCtrl    : OUT std_logic_vector(3 DOWNTO 0));  
END alu_control;  
  
ARCHITECTURE behavioral OF alu_control IS  
BEGIN  
    PROCESS (ALUOp, Instr)  
    BEGIN  
        IF ALUOp = "10" THEN -- addi  
            ALUCtrl <= "0001";  
        ELSIF ALUOp = "01" THEN  
            ALUCtrl <= "1000"; -- bne  
        ELSIF ALUOp = "00" THEN  
            CASE Instr IS  
                WHEN "100000" => -- add  
                    ALUCtrl <= "0100";  
                WHEN "100010" => -- sub  
                    ALUCtrl <= "0101";  
                WHEN others => -- default  
                    ALUCtrl <= "1110";  
            END CASE;  
        END IF;  
    END PROCESS;  
END behavioral;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```

        END CASE;

    ELSE                                     -- default

        ALUCtrl <= "1111";

    END IF;

END PROCESS;

END behavioral;
```

alu_control_tb.vhd

[illegible]

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
BEGIN

    -- addi
    ALUOp_tb <= "10";
    Instr_tb <= "001000";
    wait for 10 ns;

    -- bne
    ALUOp_tb <= "01";
    Instr_tb <= "000000";
    wait for 10 ns;

    -- add
    ALUOp_tb <= "00";
    Instr_tb <= "100000";
    wait for 10 ns;

    -- sub
    ALUOp_tb <= "00";
    Instr_tb <= "100010";
    wait for 10 ns;

    -- default
    ALUOp_tb <= "00";
    Instr_tb <= "111111";
    wait for 10 ns;

    -- default
    ALUOp_tb <= "11";
    wait for 10 ns;

END PROCESS;

END testbench;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

7. PC

program_counter.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY program_counter IS
    PORT (
        PC_input   : IN std_logic_vector(31 DOWNT0 0);
        PC_clock   : IN std_logic;
        PC_reset   : IN std_logic;
        PC_output  : OUT std_logic_vector(31 DOWNT0 0));
END program_counter;

ARCHITECTURE behavioral OF program_counter IS
BEGIN
    PROCESS(PC_reset, PC_clock)
    BEGIN
        IF PC_reset = '1' THEN
            PC_output <= "00000000000000000000000000000000";
        END IF;
        IF rising_edge(PC_clock) THEN
            PC_output <= PC_input;
        END IF;
    END PROCESS;
END behavioral;
```

program_counter_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY program_counter_tb IS
END program_counter_tb;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ARCHITECTURE testbench OF program_counter_tb IS

COMPONENT program_counter

PORT (

PC_input : IN std_logic_vector(31 DOWNT0 0);

PC_clock : IN std_logic;

PC_reset : IN std_logic;

PC_output : OUT std_logic_vector(31 DOWNT0 0));

END COMPONENT;

SIGNAL PC_clock_tb, PC_reset_tb : std_logic;

SIGNAL PC_input_tb, PC_output_tb : std_logic_vector(31 DOWNT0 0);

BEGIN

Comp_Connection : program_counter PORT MAP (PC_input => PC_input_tb,
PC_clock => PC_clock_tb,
PC_reset => PC_reset_tb,
PC_output => PC_output_tb);

clock_process : PROCESS

BEGIN

PC_clock_tb <= '0';

wait for 5 ns;

PC_clock_tb <= '1';

wait for 5 ns;

END PROCESS;

stim_process : PROCESS

BEGIN

PC_reset_tb <= '1';

PC_input_tb <= "00000000000000000000000000000000";

wait for 10 ns;

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
PC_reset_tb <= '0';
wait for 10 ns;

PC_input_tb <= "00000000000000000000000000000001";
wait for 10 ns;

PC_input_tb <= "00000000000000000000000000000010";
wait for 10 ns;

PC_input_tb <= "00000000000000000000000000000011";
wait for 10 ns;

END PROCESS;

END testbench;
```

8. 5-πλός πολυπλέκτης 2-σε-1

mux_5x_2to1.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY mux_5x_2to1 IS
    PORT (
        Mux5_input1, Mux5_input2 : IN std_logic_vector(4 DOWNTO 0);
        Mux5_S                     : IN std_logic;
        Mux5_output                : OUT std_logic_vector(4 DOWNTO 0));
END mux_5x_2to1;

ARCHITECTURE dataflow OF mux_5x_2to1 IS
BEGIN
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
Mux5_output <= Mux5_input1 WHEN Mux5_S = '1' ELSE
                Mux5_input2 WHEN Mux5_S = '0';

END dataflow;
```

mux_5x_2to1_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY mux_5x_2to1_tb IS
END mux_5x_2to1_tb;

ARCHITECTURE testbench OF mux_5x_2to1_tb IS

    COMPONENT mux_5x_2to1
    PORT (
        Mux5_input1, Mux5_input2    : IN std_logic_vector(4 DOWNTO 0);
        Mux5_S                      : IN std_logic;
        Mux5_output                 : OUT std_logic_vector(4 DOWNTO 0));
    END COMPONENT;

    SIGNAL Mux5_input1_tb, Mux5_input2_tb, Mux5_output_tb :
std_logic_vector(4 DOWNTO 0);
    SIGNAL Mux5_S_tb : std_logic;

BEGIN

    Comp_Connection : mux_5x_2to1 PORT MAP (Mux5_input1 => Mux5_input1_tb,
                                             Mux5_input2 => Mux5_input2_tb,
                                             Mux5_S => Mux5_S_tb,
                                             Mux5_output => Mux5_output_tb);

PROCESS
BEGIN
    Mux5_S_tb <= '0';
    wait for 10 ns;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
Mux5_s_tb <= '1';  
wait for 10 ns;  
END PROCESS;  
  
PROCESS  
BEGIN  
    Mux5_input1_tb <= "00000";  
    Mux5_input2_tb <= "11111";  
    wait for 10 ns;  
END PROCESS;  
END testbench;
```

9. Μονάδα επέκτασης προσήμου 16-σε-32

sign_extend_16to32.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
  
ENTITY sign_extend_16to32 IS  
    PORT (  
        SignExtend_input  : IN  std_logic_vector(15 DOWNTO 0);  
        SignExtend_output  : OUT std_logic_vector(31 DOWNTO 0));  
END sign_extend_16to32;  
  
ARCHITECTURE dataflow OF sign_extend_16to32 IS  
    SIGNAL ones : std_logic_vector(15 DOWNTO 0) := (OTHERS => '1');  
    SIGNAL zeros : std_logic_vector(15 DOWNTO 0) := (OTHERS => '0');  
BEGIN
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
        SignExtend_output <= ones & SignExtend_input WHEN  
SignExtend_input(15) = '1' ELSE  
        zeros & SignExtend_input WHEN SignExtend_input(15) = '0';  
END dataflow;
```

sign_extend_16to32_tb.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
  
ENTITY sign_extend_16to32_tb IS  
END sign_extend_16to32_tb;  
  
ARCHITECTURE testbench OF sign_extend_16to32_tb IS  
  
    COMPONENT sign_extend_16to32  
    PORT (  
        SignExtend_input  : IN  std_logic_vector(15 DOWNTO 0);  
        SignExtend_output  : OUT std_logic_vector(31 DOWNTO 0));  
    END COMPONENT;  
  
    SIGNAL SignExtend_input_tb  : std_logic_vector(15 DOWNTO 0);  
    SIGNAL SignExtend_output_tb : std_logic_vector(31 DOWNTO 0);  
  
BEGIN  
    Comp_Connection : sign_extend_16to32 PORT MAP (SignExtend_input =>  
SignExtend_input_tb,  
                                                    SignExtend_output =>  
SignExtend_output_tb);  
  
PROCESS  
    BEGIN  
        SignExtend_input_tb <= "0101010101010101";  
        wait for 10 ns;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
SignExtend_input_tb <= "1010101010101010";  
wait for 10 ns;  
END PROCESS;  
END testbench;
```

10, 11, 12. 32-πλος πολυπλέκτης 2-σε-1

mux_32x_2to1.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
  
ENTITY mux_32x_2to1 IS  
    PORT (  
        Mux32_input1, Mux32_input2 : IN std_logic_vector(31 DOWNTO 0);  
        Mux32_S                     : IN std_logic;  
        Mux32_output                : OUT std_logic_vector(31 DOWNTO  
0));  
END mux_32x_2to1;  
  
ARCHITECTURE dataflow OF mux_32x_2to1 IS  
BEGIN  
    Mux32_output <= Mux32_input1 WHEN Mux32_S = '1' ELSE  
        Mux32_input2 WHEN Mux32_S = '0';  
END dataflow;
```

mux_32x_2to1_tb.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
  
ENTITY mux_32x_2to1_tb IS  
END mux_32x_2to1_tb;  
  
ARCHITECTURE testbench OF mux_32x_2to1_tb IS
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

[illegible]

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

END testbench;

13. Μονάδα Ολίσθησης αριστερά κατά 2 (32-bits)

shifter_2left.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY shifter_2left IS
    PORT (
        Shifter_input      : IN  std_logic_vector(31 DOWNTO 0);
        Shifter_output     : OUT std_logic_vector(31 DOWNTO 0));
END shifter_2left;

ARCHITECTURE behavioral OF shifter_2left IS
    SIGNAL tmp : unsigned(31 DOWNTO 0);
BEGIN
    tmp <= to_unsigned(to_integer(signed(Shifter_input)), tmp'length) SLL
    2;

    Shifter_output <= std_logic_vector(to_signed(to_integer(tmp),
    Shifter_output'length));
END behavioral;
```

shifter_2left_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY shifter_2left_tb IS
END shifter_2left_tb;

ARCHITECTURE testbench OF shifter_2left_tb IS

    COMPONENT shifter_2left
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
PORT (  
    Shifter_input      : IN  std_logic_vector(31 DOWNTO 0);  
    Shifter_output     : OUT std_logic_vector(31 DOWNTO 0));  
END COMPONENT;  
  
SIGNAL Shifter_input_tb, Shifter_output_tb : std_logic_vector(31 DOWNTO  
0);  
  
BEGIN  
    Comp_Connection : shifter_2left PORT MAP (Shifter_input =>  
Shifter_input_tb,  
                                                Shifter_output =>  
Shifter_output_tb);  
PROCESS  
    BEGIN  
        Shifter_input_tb <= (others => '0');  
        wait for 10 ns;  
  
        Shifter_input_tb <= (others => '1');  
        wait for 10 ns;  
  
        Shifter_input_tb <= "11001100110011001100110011001100";  
        wait for 10 ns;  
    END PROCESS;  
END testbench;
```

14. Αθροιστής 32 bits

pc_adder.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
USE ieee.numeric_std.all;  
  
ENTITY pc_adder IS
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
PORT (  
    previousCmdAddr : IN std_logic_vector(31 DOWNTO 0);  
    nextCmdAddr      : OUT std_logic_vector(31 DOWNTO 0));  
END pc_adder;  
  
ARCHITECTURE dataflow OF pc_adder IS  
    SIGNAL sum : unsigned(31 DOWNTO 0);  
    BEGIN  
        sum <= unsigned(previousCmdAddr) + 1;  
        nextCmdAddr <= std_logic_vector(sum);  
    END dataflow;
```

pc_adder_tb.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
  
ENTITY pc_adder_tb IS  
END pc_adder_tb;  
  
ARCHITECTURE testbench OF pc_adder_tb IS  
  
    COMPONENT pc_adder  
        PORT (  
            previousCmdAddr : IN std_logic_vector(31 DOWNTO 0);  
            nextCmdAddr      : OUT std_logic_vector(31 DOWNTO 0));  
    END COMPONENT;  
  
    SIGNAL previousCmdAddr_tb, nextCmdAddr_tb : std_logic_vector(31 DOWNTO 0);  
  
    BEGIN  
        Comp_Connection : pc_adder PORT MAP (previousCmdAddr =>  
previousCmdAddr_tb,  
nextCmdAddr => nextCmdAddr_tb);
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
PROCESS
BEGIN
    previousCmdAddr_tb <= "00000000000000000000000000000000";
    wait for 10 ns;

    previousCmdAddr_tb <= "00000000000000000000000000000001";
    wait for 10 ns;

    previousCmdAddr_tb <= "00000000000000000000000000000010";
    wait for 10 ns;

    previousCmdAddr_tb <= "00000000000000000000000000000011";
    wait for 10 ns;

    previousCmdAddr_tb <= "11111111111111111111111111111111";
    wait for 10 ns;

    previousCmdAddr_tb <= "11111111111111111111111111111111";
    wait for 10 ns;

END PROCESS;

END testbench;
```

15. Αθροιστής 32 bits

fullAdder_32bit.vhd

```
LIBRARY ieee;

USE ieee.std_logic_1164.all;

USE ieee.std_logic_unsigned.all;

USE ieee.numeric_std.all;

ENTITY fullAdder_32bit IS
    PORT (
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
    Adder_input1, Adder_input2  : IN std_logic_vector(31 DOWNTO 0);
    Adder_output                 : OUT std_logic_vector(31 DOWNTO 0)
);
END fullAdder_32bit;

ARCHITECTURE dataflow OF fullAdder_32bit IS
BEGIN
    Adder_output <= Adder_input1 + Adder_input2;
END dataflow;
```

fullAdder_32bit_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY fullAdder_32bit_tb IS
END fullAdder_32bit_tb;

ARCHITECTURE testbench OF fullAdder_32bit_tb IS

    COMPONENT fullAdder_32bit
    PORT (
        Adder_input1, Adder_input2  : IN std_logic_vector(31 DOWNTO 0);
        Adder_Cin                   : IN std_logic_vector(0 DOWNTO 0);
        Adder_output                 : OUT std_logic_vector(31 DOWNTO
0);
        Adder_Cout                   : OUT std_logic);
    END COMPONENT;

    SIGNAL Adder_input1_tb, Adder_input2_tb, Adder_output_tb :
std_logic_vector(31 DOWNTO 0);

    SIGNAL Adder_Cin_tb : std_logic_vector(0 DOWNTO 0);

    SIGNAL Adder_Cout_tb : std_logic;

BEGIN
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
Comp_Connection : fullAdder_32bit PORT MAP (Adder_input1 =>
Adder_input1_tb,

Adder_input2 =>
Adder_input2_tb,

Adder_Cin => Adder_Cin_tb,
Adder_output =>
Adder_output_tb,

Adder_Cout =>
Adder_Cout_tb);

PROCESS
BEGIN
    Adder_input1_tb <= "01010101010101010101010101010101";
    Adder_input2_tb <= "00110011001100110011001100110011";
    Adder_Cin_tb <= "1";
    wait for 10 ns;

    Adder_input1_tb <= "11000011001100110011001100110011";
    Adder_input2_tb <= "01111111111111111111111111111111";
    Adder_Cin_tb <= "0";
    wait for 10 ns;
END PROCESS;
END testbench;
```

Πύλη AND 2 εισόδων

and2_gate.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY and_2gate IS
    PORT (
        Branch, Zero : IN std_logic;
        AND_output : OUT std_logic);
END and_2gate;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
ARCHITECTURE dataflow OF and_2gate IS
```

```
BEGIN
```

```
    AND_output <= Branch AND Zero;
```

```
END dataflow;
```

and2_gate_tb.vhd

```
LIBRARY ieee;
```

```
USE ieee.std_logic_1164.all;
```

```
ENTITY and_2gate_tb IS
```

```
END and_2gate_tb;
```

```
ARCHITECTURE testbench OF and_2gate_tb IS
```

```
    COMPONENT and_2gate
```

```
    PORT (
```

```
        Branch, Zero : IN std_logic;
```

```
        AND_output    : OUT std_logic);
```

```
END COMPONENT;
```

```
SIGNAL Branch_tb, Zero_tb, AND_output_tb : std_logic;
```

```
BEGIN
```

```
    Comp_Connection : and_2gate PORT MAP (Branch => Branch_tb,
```

```
                                             Zero => Zero_tb,
```

```
                                             AND_output => AND_output_tb);
```

```
PROCESS
```

```
    BEGIN
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
Branch_tb <= '0'; Zero_tb <= '0';  
wait for 10 ns;  
Branch_tb <= '1'; Zero_tb <= '0';  
wait for 10 ns;  
Branch_tb <= '0'; Zero_tb <= '1';  
wait for 10 ns;  
Branch_tb <= '1'; Zero_tb <= '1';  
wait for 10 ns;  
END PROCESS;  
END testbench;
```

MIPS

19390005_ATHANASIOU_02_MIPS.vhd

```
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
USE ieee.std_logic_signed.all;  
  
ENTITY MIPS IS  
  PORT (  
    MIPS_clock : IN std_logic;  
    MIPS_reset : IN std_logic;  
    PC_output_MIPS : OUT std_logic_vector(31 DOWNTO 0);  
    ALU_output_MIPS : OUT std_logic_vector(31 DOWNTO 0);  
    zeroSig_MIPS : OUT std_logic;  
    readData1_MIPS : OUT std_logic_vector(31 DOWNTO 0);  
    readData2_MIPS : OUT std_logic_vector(31 DOWNTO 0);  
    Branch_MIPS : OUT std_logic;  
    instrOut_MIPS : OUT std_logic_vector(31 DOWNTO 0);  
    writeData_MIPS : OUT std_logic_vector(31 DOWNTO 0);  
    MemWrite_MIPS : OUT std_logic;  
    RegWrite_MIPS : OUT std_logic;  
  );
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
END MIPS;
```

```
ARCHITECTURE structural OF MIPS IS
```

```
COMPONENT alu
```

```
PORT (
```

```
    operationSig    : IN std_logic_vector(3 DOWNTO 0);  
    ALU_input1      : IN std_logic_vector(31 DOWNTO 0);  
    ALU_input2      : IN std_logic_vector(31 DOWNTO 0);  
    ALU_output      : OUT std_logic_vector(31 DOWNTO 0);  
    zeroSig         : OUT std_logic);
```

```
END COMPONENT;
```

```
COMPONENT register_file
```

```
PORT (
```

```
    RF_clock        : IN std_logic;  
    RF_reset        : IN std_logic;  
    readReg1        : IN std_logic_vector(4 DOWNTO 0);  
    readReg2        : IN std_logic_vector(4 DOWNTO 0);  
    writeRegIn      : IN std_logic_vector(4 DOWNTO 0);  
    writeData       : IN std_logic_vector(31 DOWNTO 0);  
    RegWrite        : IN std_logic;  
    readData1       : OUT std_logic_vector(31 DOWNTO 0);  
    readData2       : OUT std_logic_vector(31 DOWNTO 0));
```

```
END COMPONENT;
```

```
COMPONENT data_memory
```

```
PORT (
```

```
    writeData       : IN std_logic_vector(31 DOWNTO 0);  
    addr            : IN std_logic_vector(31 DOWNTO 0);  
    MemRead         : IN std_logic;  
    MemWrite        : IN std_logic;  
    readData        : OUT std_logic_vector(31 DOWNTO 0));
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
END COMPONENT;
```

```
COMPONENT instructions_memory
```

```
PORT (
```

```
    readAddr      : IN std_logic_vector(31 DOWNTO 0);
```

```
    instrOut       : OUT std_logic_vector(31 DOWNTO 0));
```

```
END COMPONENT;
```

```
COMPONENT control_unit
```

```
PORT (
```

```
    CU_clock       : IN std_logic;
```

```
    Instr          : IN std_logic_vector(5 DOWNTO 0);
```

```
    RegDst         : OUT std_logic;
```

```
    Branch         : OUT std_logic;
```

```
    MemRead        : OUT std_logic;
```

```
    MemtoReg       : OUT std_logic;
```

```
    ALUOp          : OUT std_logic_vector(1 DOWNTO 0);
```

```
    MemWrite       : OUT std_logic;
```

```
    ALUSrc         : OUT std_logic;
```

```
    RegWrite       : OUT std_logic);
```

```
END COMPONENT;
```

```
COMPONENT alu_control
```

```
PORT (
```

```
    ALUOp          : IN std_logic_vector(1 DOWNTO 0);
```

```
    Instr          : IN std_logic_vector(5 DOWNTO 0);
```

```
    ALUCtrl        : OUT std_logic_vector(3 DOWNTO 0));
```

```
END COMPONENT;
```

```
COMPONENT program_counter
```

```
PORT (
```

```
    PC_input       : IN std_logic_vector(31 DOWNTO 0);
```

```
    PC_clock       : IN std_logic;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
PC_reset   : IN std_logic;

PC_output  : OUT std_logic_vector(31 DOWNTO 0));

END COMPONENT;


COMPONENT pc_adder

PORT (

    previousCmdAddr : IN std_logic_vector(31 DOWNTO 0);
    nextCmdAddr      : OUT std_logic_vector(31 DOWNTO 0));

END COMPONENT;

COMPONENT mux_5x_2to1

PORT (

    Mux5_input1, Mux5_input2 : IN std_logic_vector(4 DOWNTO 0);
    Mux5_S                  : IN std_logic;
    Mux5_output              : OUT std_logic_vector(4 DOWNTO 0));

END COMPONENT;


COMPONENT sign_extend_16to32

PORT (

    SignExtend_input   : IN  std_logic_vector(15 DOWNTO 0);
    SignExtend_output  : OUT std_logic_vector(31 DOWNTO 0));

END COMPONENT;


COMPONENT mux_32x_2to1

PORT (

    Mux32_input1, Mux32_input2 : IN std_logic_vector(31 DOWNTO 0);
    Mux32_S                  : IN std_logic;
    Mux32_output              : OUT std_logic_vector(31 DOWNTO 0));

END COMPONENT;


COMPONENT shifter_2left

PORT (

    Shifter_input   : IN  std_logic_vector(31 DOWNTO 0);
    Shifter_output  : OUT std_logic_vector(31 DOWNTO 0));
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
END COMPONENT;
```

```
COMPONENT fullAdder_32bit
```

```
PORT (
```

```
    Adder_input1, Adder_input2 : IN std_logic_vector(31 DOWNTO 0);
```

```
    Adder_output                : OUT std_logic_vector(31 DOWNTO 0));
```

```
END COMPONENT;
```

```
COMPONENT and_2gate
```

```
PORT (
```

```
    Branch, Zero : IN std_logic;
```

```
    AND_output    : OUT std_logic);
```

```
END COMPONENT;
```

```
-- ALU out signals
```

```
SIGNAL ALU_output_Sig : std_logic_vector(31 DOWNTO 0);
```

```
SIGNAL zeroSig_Sig    : std_logic;
```

```
-- Register File out signals
```

```
SIGNAL readData1_Sig : std_logic_vector(31 DOWNTO 0);
```

```
SIGNAL readData2_Sig : std_logic_vector(31 DOWNTO 0);
```

```
SIGNAL writeData_Sig : std_logic_vector(31 DOWNTO 0);
```

```
-- Data Memory out signals
```

```
SIGNAL readDataMemory_Sig : std_logic_vector(31 DOWNTO 0);
```

```
-- Instructions Memory out signals
```

```
SIGNAL instrOut_Sig : std_logic_vector(31 DOWNTO 0);
```

```
-- Control Unit
```

```
SIGNAL Instr_Sig      : std_logic_vector(5 DOWNTO 0);
```

```
SIGNAL RegDst_Sig     : std_logic;
```

```
SIGNAL Branch_Sig     : std_logic;
```

```
SIGNAL MemRead_Sig    : std_logic;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
SIGNAL MemtoReg_Sig  : std_logic;
SIGNAL ALUOp_Sig      : std_logic_vector(1 DOWNTO 0);
SIGNAL MemWrite_Sig  : std_logic;
SIGNAL ALUSrc_Sig     : std_logic;
SIGNAL RegWrite_Sig  : std_logic;

-- ALU Control out signals
SIGNAL ALUCtrl_Sig : std_logic_vector(3 DOWNTO 0);

-- Program Counter out signals
SIGNAL PC_output_Sig : std_logic_vector(31 DOWNTO 0);

-- PC Adder out signals
SIGNAL nextCmdAddr_Sig : std_logic_vector(31 DOWNTO 0);

-- Mux 5x 2-to-1 out signals
SIGNAL Mux5_output_Register : std_logic_vector(4 DOWNTO 0);

-- Sign Extend 16-to-32 out signals
SIGNAL SignExtend_output_Sig : std_logic_vector(31 DOWNTO 0);

-- Mux 32x 2-to-1 out signals
SIGNAL Mux32_output_PC      : std_logic_vector(31 DOWNTO 0);
SIGNAL Mux32_output_ALU     : std_logic_vector(31 DOWNTO 0);
SIGNAL Mux32_output_Register : std_logic_vector(31 DOWNTO 0);

-- Shifter 2 Left out signals
SIGNAL Shifter_output_Sig : std_logic_vector(31 DOWNTO 0);

-- Full Adder 32-bit out signals
SIGNAL Adder_output_Sig : std_logic_vector(31 DOWNTO 0);
SIGNAL Adder_Cout_Sig   : std_logic;
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

[illegible]

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

[illegible]

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
DataMemory : data_memory PORT MAP (writeData => readData2_Sig,
                                     addr => ALU_output_Sig,
                                     MemRead => MemRead_Sig,
                                     MemWrite => MemWrite_Sig,
                                     readData => readDataMemory_Sig);

Mux5x2to1_Register : mux_5x_2to1 PORT MAP (Mux5_input1 =>
instrOut_Sig(20 DOWNT0 16),
                                     Mux5_input2 =>
instrOut_Sig(15 DOWNT0 11),
                                     Mux5_S => RegDst_Sig,
                                     Mux5_output =>
Mux5_output_Register);

Mux32x2to1_ALU : mux_32x_2to1 PORT MAP (Mux32_input1 => readData2_Sig,
                                     Mux32_input2 =>
SignExtend_output_Sig,
                                     Mux32_S => ALUSrc_Sig,
                                     Mux32_output =>
Mux32_output_ALU);

Mux32x2to1_Register : mux_32x_2to1 PORT MAP (Mux32_input1 =>
readDataMemory_Sig,
                                     Mux32_input2 =>
ALU_output_Sig,
                                     Mux32_S => MemtoReg_Sig,
                                     Mux32_output =>
Mux32_output_Register);

Mux32x2to1_PC : mux_32x_2to1 PORT MAP (Mux32_input1 => nextCmdAddr_Sig,
                                     Mux32_input2 => Adder_output_Sig,
                                     Mux32_S => AND_output_Sig,
                                     Mux32_output => Mux32_output_PC);

SignExtend16to32 : sign_extend_16to32 PORT MAP (SignExtend_input =>
instrOut_Sig(15 DOWNT0 0),
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
SignExtend_output =>
SignExtend_output_Sig);

AND2gate : and_2gate PORT MAP (Branch => Branch_Sig,
                                Zero => zeroSig_Sig,
                                AND_output => AND_output_Sig);

Shifter2Left : shifter_2left PORT MAP (Shifter_input =>
SignExtend_output_Sig,
                                         Shifter_output =>
Shifter_output_Sig);

END structural;
```

19390005_ATHANASIOU_03_testbench.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY MIPS_tb IS
END MIPS_tb;

ARCHITECTURE testbench OF MIPS_tb IS

    COMPONENT MIPS
    PORT (
        MIPS_clock : IN std_logic;
        MIPS_reset : IN std_logic;
        PC_output_MIPS : OUT std_logic_vector(31 DOWNTO 0);
        ALU_output_MIPS : OUT std_logic_vector(31 DOWNTO 0);
        zeroSig_MIPS : OUT std_logic;
        readData1_MIPS : OUT std_logic_vector(31 DOWNTO 0);
        readData2_MIPS : OUT std_logic_vector(31 DOWNTO 0);
        Branch_MIPS : OUT std_logic;
        instrOut_MIPS : OUT std_logic_vector(31 DOWNTO 0);
        writeData_MIPS : OUT std_logic_vector(31 DOWNTO 0);
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
MemWrite_MIPS : OUT std_logic;

RegWrite_MIPS : OUT std_logic

);

END COMPONENT;


SIGNAL MIPS_clock_tb      : std_logic := '0';
SIGNAL MIPS_reset_tb      : std_logic := '0';
SIGNAL PC_output_tb       : std_logic_vector(31 DOWNTO 0) := (others
=> '0');
SIGNAL ALU_output_tb      : std_logic_vector(31 DOWNTO 0) := (others
=> '0');
SIGNAL zeroSig_tb         : std_logic := '0';
SIGNAL readData1_tb       : std_logic_vector(31 DOWNTO 0) := (others
=> '0');
SIGNAL readData2_tb       : std_logic_vector(31 DOWNTO 0) := (others
=> '0');
SIGNAL Branch_tb          : std_logic := '0';
SIGNAL instrOut_tb : std_logic_vector(31 DOWNTO 0) := (others => '0');
SIGNAL writeData_tb       : std_logic_vector(31 DOWNTO 0) := (others =>
'0');
SIGNAL MemWrite_tb        : std_logic := '0';
SIGNAL RegWrite_tb        : std_logic := '0';


BEGIN

MIPS_testbench : MIPS      PORT MAP (

MIPS_clock      => MIPS_clock_tb,
MIPS_reset      => MIPS_reset_tb,
PC_output_MIPS  => PC_output_tb,
ALU_output_MIPS => ALU_output_tb,
zeroSig_MIPS    => zeroSig_tb,
readData1_MIPS  => readData1_tb,
readData2_MIPS  => readData2_tb,
Branch_MIPS     => Branch_tb,
instrOut_MIPS   => instrOut_tb,
writeData_MIPS  => writeData_tb,
```

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

```
MemWrite_MIPS => MemWrite_tb,  
RegWrite_MIPS => RegWrite_tb  
);
```

```
clock_process : PROCESS
```

```
    BEGIN
```

```
        MIPS_clock_tb <= '0';
```

```
        wait for 5 ns;
```

```
        MIPS_clock_tb <= '1';
```

```
        wait for 5 ns;
```

```
    END PROCESS;
```

```
reset_process : PROCESS
```

```
    BEGIN
```

```
        MIPS_reset_tb <= '1';
```

```
        wait for 10 ns;
```

```
        MIPS_reset_tb <= '0';
```

```
        wait;
```

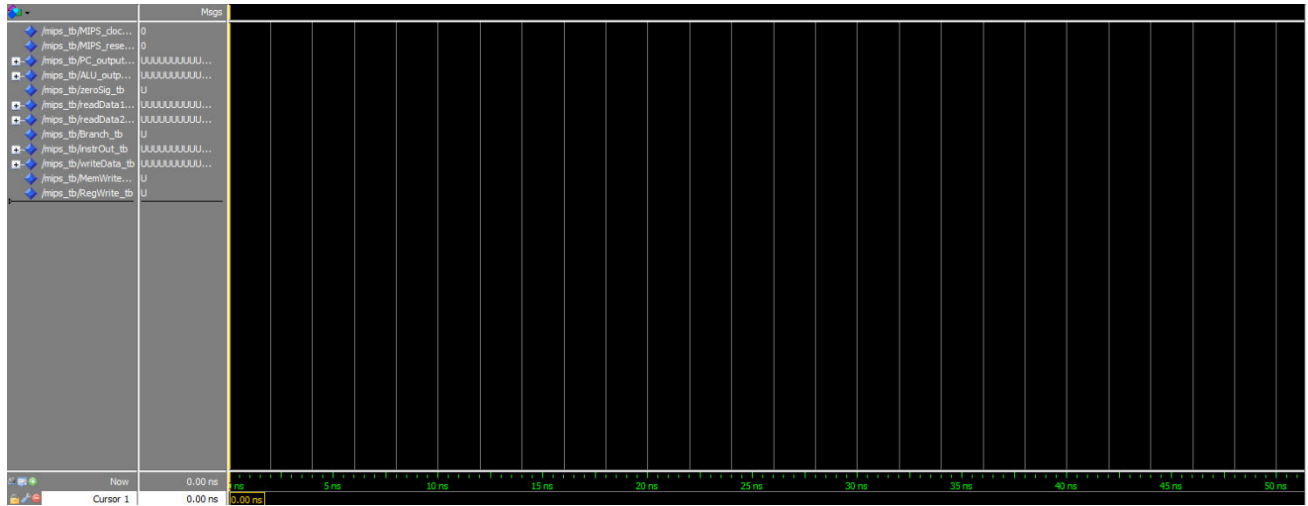
```
    END PROCESS;
```

```
END testbench;
```

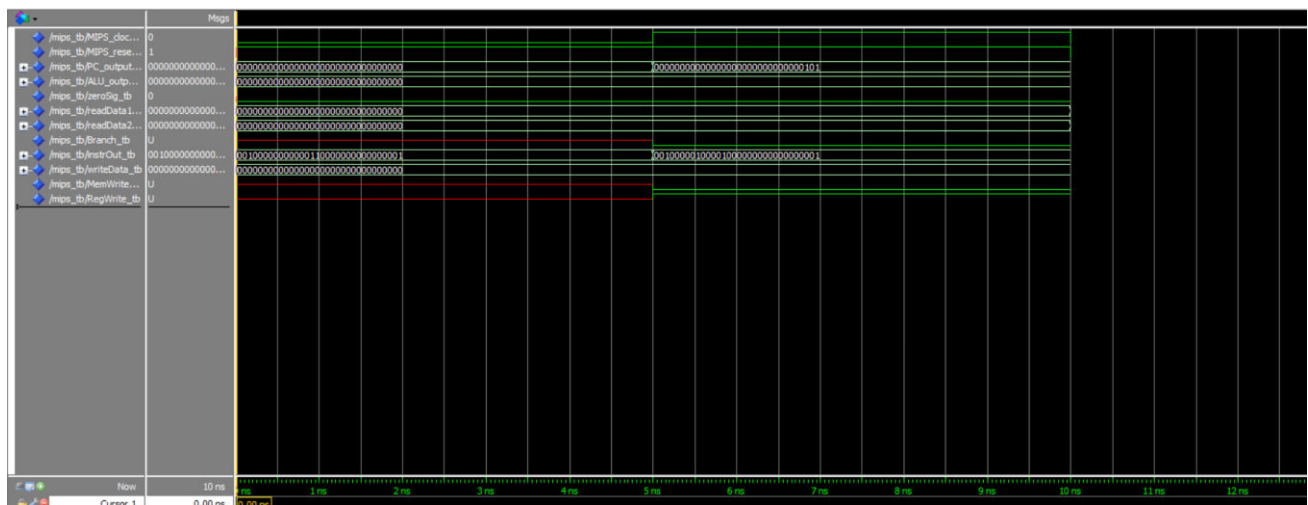
ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

3. Screenshot από την εκτέλεση

Πριν την εκτέλεση

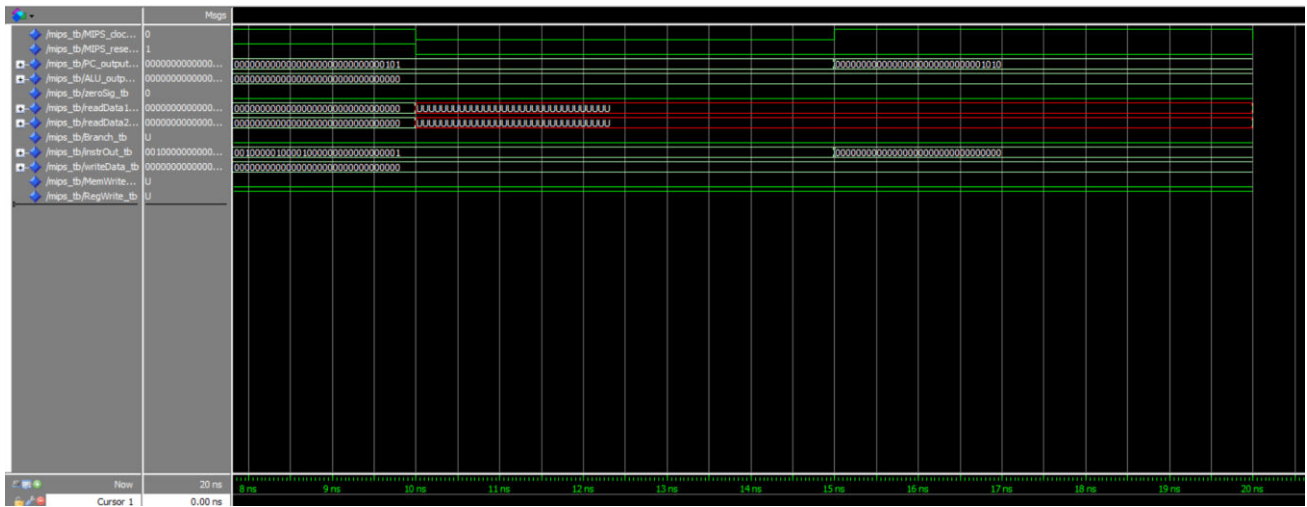


1^o clock

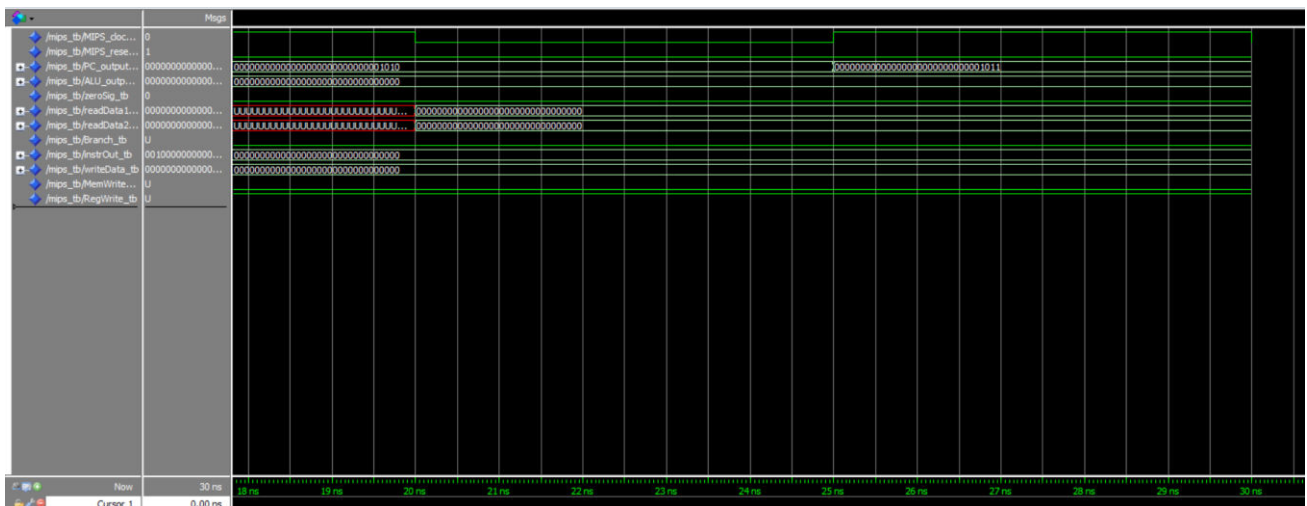


ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

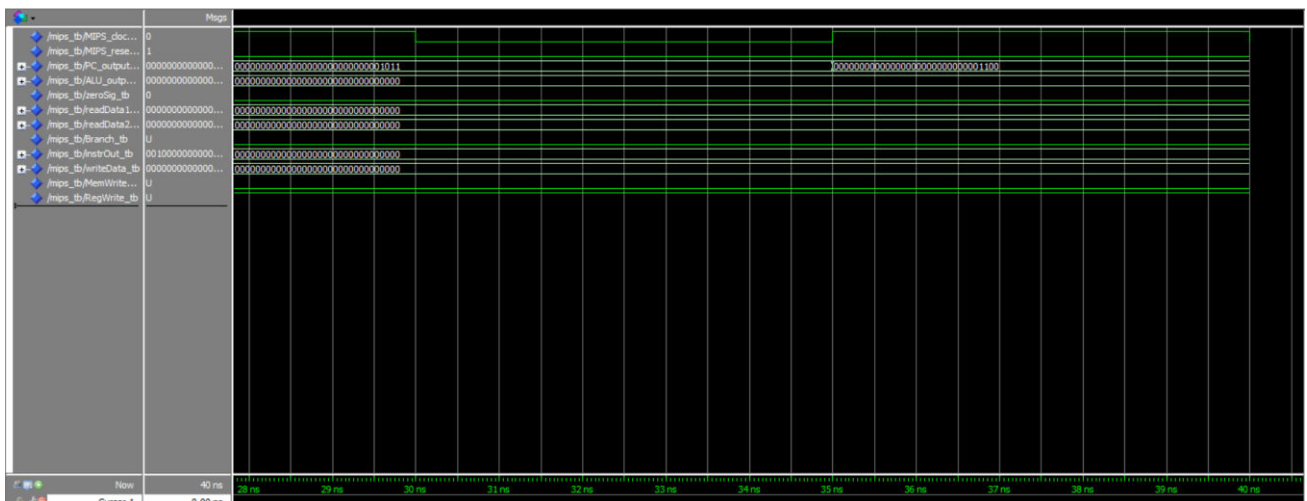
2° clock



3º clock

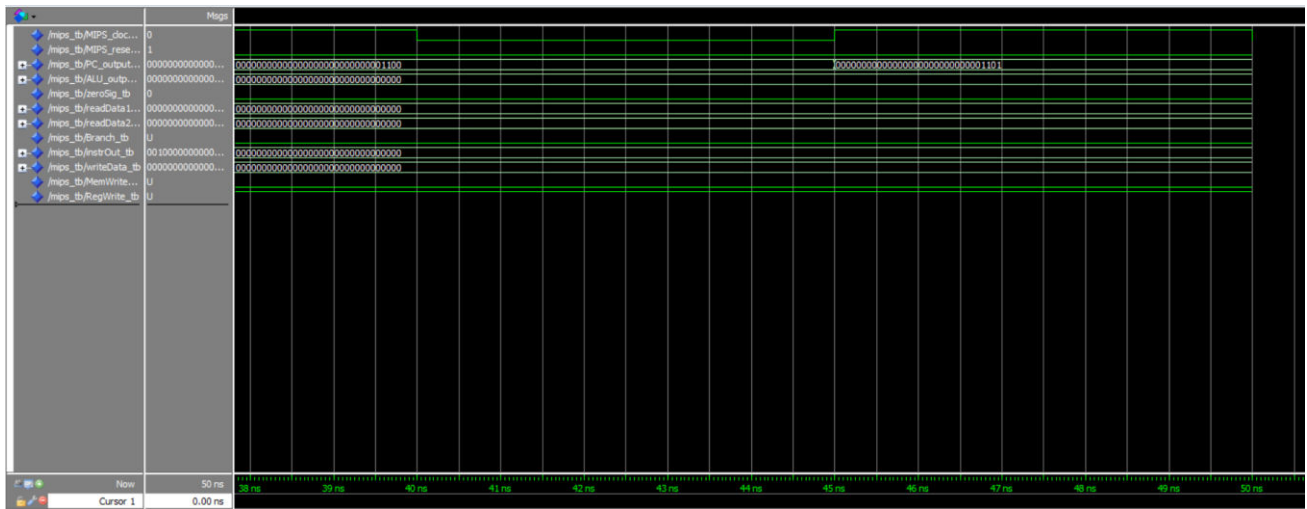


4^o clock

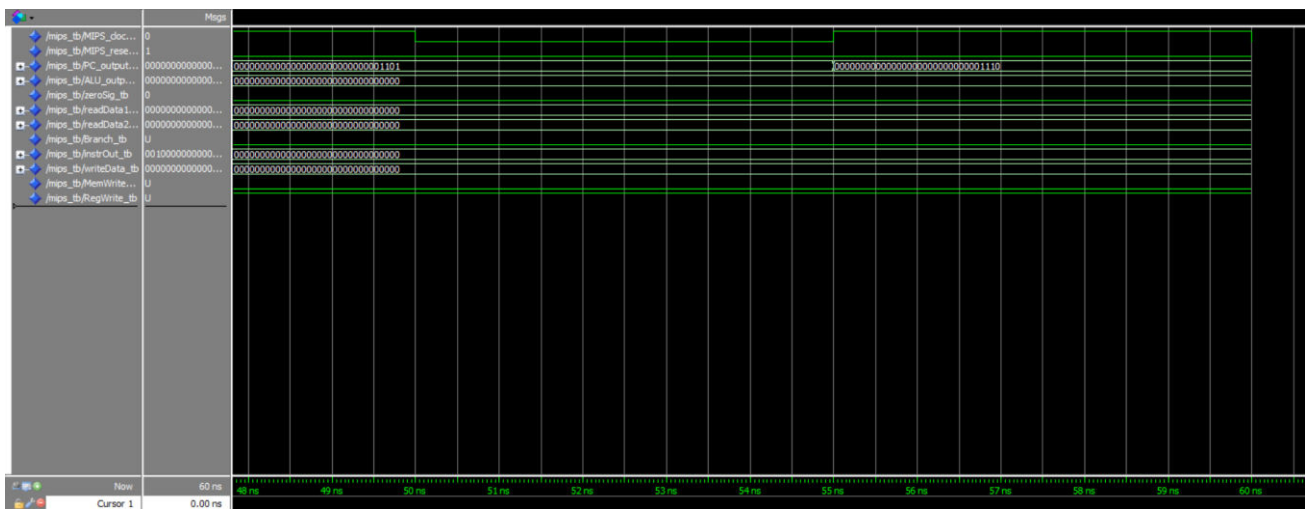


ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

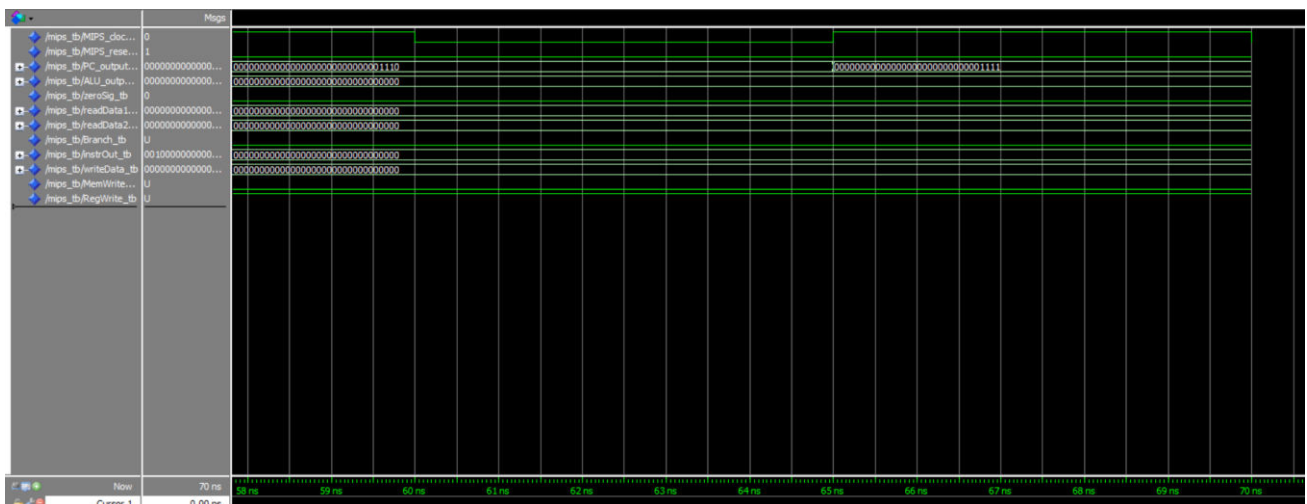
5^ο clock



6^ο clock



7^ο clock



ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ



Σας ευχαριστώ για την προσοχή σας.

